

ja

Super, dann schauen wir uns an, wie das Herzstück deiner Lösung aussehen sollte.

Da es sich bei StayAI bzw. der Testseite um eine React-Anwendung (oder ähnliche Single-Page-Application) handelt, wird das reine Suchen und Ersetzen beim Laden der Seite nicht ausreichen. React lädt und verändert DOM-Elemente ständig neu (z.B. beim Klicken auf Tabs oder Laden von Daten).

Daher nutzen wir den **MutationObserver**. Dieser überwacht das HTML-Dokument auf Veränderungen und feuert unser Skript jedes Mal ab, wenn neue Elemente auf dem Bildschirm erscheinen.

Hier ist ein gut strukturiertes Grundgerüst, das du für deine Case Study anpassen kannst:

JavaScript

```
// Wir packen alles in eine IIFE (Immediately Invoked Function Expression),
// damit wir keine globalen Variablen stören.
(function() {
  'use strict';

  // 1. Wörterbuch für die fehlenden Übersetzungen anlegen
  const translations = {
    "Subtotal": "Zwischensumme",
    "Shipping": "Versand",
    "Taxes": "Steuern",
    "Total": "Gesamtsumme",
    "Manage Subscription": "Abonnement verwalten",
    "Next delivery": "Nächste Lieferung"
    // Füge hier weitere fehlende Begriffe hinzu, die du auf der Testseite findest
  };

  // 2. Funktion zum Ersetzen von Text, Währung und Datum
  function processTextNode(node) {
    let text = node.nodeValue;
    let modified = false;

    // A. Übersetzungen anwenden
    for (const [english, german] of Object.entries(translations)) {
      // Wir suchen exakt nach dem Wort (Case-sensitive)
      if (text.includes(english)) {
```

```

    text = text.replace(new RegExp(english, 'g'), german);
    modified = true;
  }
}

```

```

// B. Währungsformat anpassen (Beispiel: $15.00 -> 15,00 €)
// Sucht nach einem $ gefolgt von Zahlen, evtl. einem Punkt und zwei Dezimalstellen
const currencyRegex = /\$(\d+)\.(\d{2})/g;
if (currencyRegex.test(text)) {
  text = text.replace(currencyRegex, '$1,$2 €');
  modified = true;
}

```

```

// C. Datumsformat anpassen (Beispiel US-Format: 12/31/2026 -> 31.12.2026)
// Geht davon aus, dass das Format MM/DD/YYYY ist.
const dateRegex = /\b(\d{1,2})\/(\d{1,2})\/(\d{4})\b/g;
if (dateRegex.test(text)) {
  // $1 = Monat, $2 = Tag, $3 = Jahr
  text = text.replace(dateRegex, (match, month, day, year) => {
    // Fügt führende Nullen hinzu, falls gewünscht
    const padDay = day.padStart(2, '0');
    const padMonth = month.padStart(2, '0');
    return `${padDay}.${padMonth}.${year}`;
  });
  modified = true;
}

```

```

// Nur das DOM aktualisieren, wenn sich wirklich etwas geändert hat (Performance!)
if (modified) {
  node.nodeValue = text;
}
}

```

```

// 3. Rekursive Funktion, die alle Text-Nodes in einem Element durchgeht
function walkDOM(node) {
  // nodeType 3 ist ein Text-Node. Das ist wichtig, da wir keine HTML-Tags
  // kaputtmachen wollen (wie <div class="Subtotal">), sondern nur den Text.
  if (node.nodeType === 3) {
    // Skripte und Styles ignorieren
    if (node.parentNode && (node.parentNode.tagName === 'SCRIPT' ||
node.parentNode.tagName === 'STYLE')) {
      return;
    }
  }
}

```

```

    // Leere Text-Nodes überspringen
    if (node.nodeValue.trim() !== "") {
        processTextNode(node);
    }
} else {
    // Falls es kein Text-Node ist, durchsuchen wir die Kinder-Elemente
    for (let i = 0; i < node.childNodes.length; i++) {
        walkDOM(node.childNodes[i]);
    }
}
}

// 4. Initialer Lauf beim Laden des Skripts
walkDOM(document.body);

// 5. MutationObserver einrichten, um zukünftige Änderungen abzufangen
const observer = new MutationObserver((mutations) => {
    mutations.forEach((mutation) => {
        // Wir prüfen alle neu hinzugefügten Nodes
        if (mutation.addedNodes && mutation.addedNodes.length > 0) {
            mutation.addedNodes.forEach((newNode) => {
                walkDOM(newNode);
            });
        }
    });
});

// Observer starten: Wir überwachen das gesamte Body-Element und alle Kind-Elemente
observer.observe(document.body, {
    childList: true,
    subtree: true,
    // characterData: true // (Optional, falls sich Textinhalte direkt ändern, ohne dass neue Nodes erstellt
    // werden)
});

console.log("StayAI Translation & Formatting Script geladen.");

})();

```

Wie du dieses Skript für die Abgabe nutzt:

1. **Testen & Anpassen:** Öffne die in der Case Study genannte Lovable-URL in Chrome. Drücke F12, um die Entwicklertools zu öffnen, wechsele in den Tab **Console (Konsole)**,

füge den gesamten Code von oben ein und drücke Enter. Beobachte, ob sich die Texte ändern.

2. **Regex prüfen:** Die regulären Ausdrücke (currencyRegex und dateRegex) in meinem Beispiel gehen von \$15.00 und MM/DD/YYYY aus. Falls die Testseite z. B. 15.00 USD oder YYYY-MM-DD verwendet, musst du die Regex im Skript entsprechend anpassen. Das ist der Teil, bei dem du im Termin zeigen kannst, dass du verstehst, was der Code macht.
3. **Auf GitHub veröffentlichen:** Erstelle eine einfache Datei namens stayai-translator.js, füge den fertigen und getesteten Code ein und lade sie in dein Repo hoch. Du kannst zusätzlich eine kurze README.md schreiben, in der du erklärst, dass dies über die Custom-JS-Einstellungen der Shopify App injiziert wird.

Bereite dich für das Gespräch gut darauf vor, **warum** du den MutationObserver (Schritt 5) und die Überprüfung auf nodeType === 3 (Schritt 3) gewählt hast. Das zeigt tiefes Verständnis für DOM-Manipulation!